

Process Implementation

CS 232 Operating Systems

Fall 2019

Sharing the CPU

- Hundreds of processes are running on a computer
 - There's only one CPU
 - How can one CPU run many processes simultaneously?
 - Time Sharing!
 - All the ready processes are in a queue
 - The OS will choose one process (A) and run it on the CPU
 - After some time it'll take back the control of CPU from process A
 - Schedule the next process (B) to run on CPU and put process A back in queue, and so on ...
-
- CPU runs much faster than humans can realize. This creates the illusion.

Two concerns

- Control
 - How to restrict a process from performing forbidden actions
 - How to get back the CPU from a running process
- Performance
 - OS code is an “overhead” from the point of view of user

Direct execution

- Fast
- Control?
 - Limited direct execution

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute call main()	
	Run main()
	Execute return from main
Free memory of process	
Remove from process list	

Figure 6.1: **Direct Execution Protocol (Without Limits)**

Control I: limiting operations

- Can't let every process do whatever it wants
- Solution: Two modes of execution (need h/w support)
 - User mode - limited access e.g. can't do I/O
 - Kernel mode - unrestricted access, can do I/O
- How should the user process do I/O then?
 - System call! trap/return-from-trap
 - syscall numbers* / trap table
 - In x86 system calls the process context is pushed on a per-process "kernel stack"

OS @ boot (kernel mode)	Hardware	
initialize trap table	remember address of... syscall handler	
OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list Allocate memory for program Load program into memory Setup user stack with argv Fill kernel stack with reg/PC return-from-trap	restore regs from kernel stack move to user mode jump to main	Run main() ... Call system call trap into OS
Handle trap Do work of syscall return-from-trap	save regs to kernel stack move to kernel mode jump to trap handler	
	restore regs from kernel stack move to user mode jump to PC after trap	... return from main trap (via <code>exit()</code>)
Free memory of process Remove from process list		

Control II: regaining control

- Cooperative approach
 - Processes yield control by performing a system call
 - Malicious or infinite process? Reboot!
- Non-cooperative approach
 - Timer interrupts
 - Save/restore context

OS @ boot (kernel mode)	Hardware	
initialize trap table	remember addresses of... syscall handler timer handler	
start interrupt timer	start timer interrupt CPU in X ms	
OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
		...
	timer interrupt save regs(A) to k-stack(A) move to kernel mode jump to trap handler	
Handle the trap Call <code>switch()</code> routine save regs(A) to <code>proc-struct(A)</code> restore regs(B) from <code>proc-struct(B)</code> switch to <code>k-stack(B)</code> return-from-trap (into B)		
	restore regs(B) from <code>k-stack(B)</code> move to user mode jump to B's PC	
		Process B
		...

Figure 6.3: Limited Direct Execution Protocol (Timer Interrupt)

```

1  # void switch(struct context **old, struct context *new);
2  #
3  # Save current register context in old
4  # and then load register context from new.
5  .globl switch
6  switch:
7      # Save old registers
8      movl 4(%esp), %eax # put old ptr into eax
9      popl 0(%eax)      # save the old IP
10     movl %esp, 4(%eax) # and stack
11     movl %ebx, 8(%eax) # and other registers
12     movl %ecx, 12(%eax)
13     movl %edx, 16(%eax)
14     movl %esi, 20(%eax)
15     movl %edi, 24(%eax)
16     movl %ebp, 28(%eax)
17
18     # Load new registers
19     movl 4(%esp), %eax # put new ptr into eax
20     movl 28(%eax), %ebp # restore other registers
21     movl 24(%eax), %edi
22     movl 20(%eax), %esi
23     movl 16(%eax), %edx
24     movl 12(%eax), %ecx
25     movl 8(%eax), %ebx
26     movl 4(%eax), %esp # stack is switched here
27     pushl 0(%eax)      # return addr put in place
28     ret               # finally return into new ctxt

```

Figure 6.4: The xv6 Context Switch Code